

리버싱 프로젝트4

RVP4 State-Coupled Virtual License System

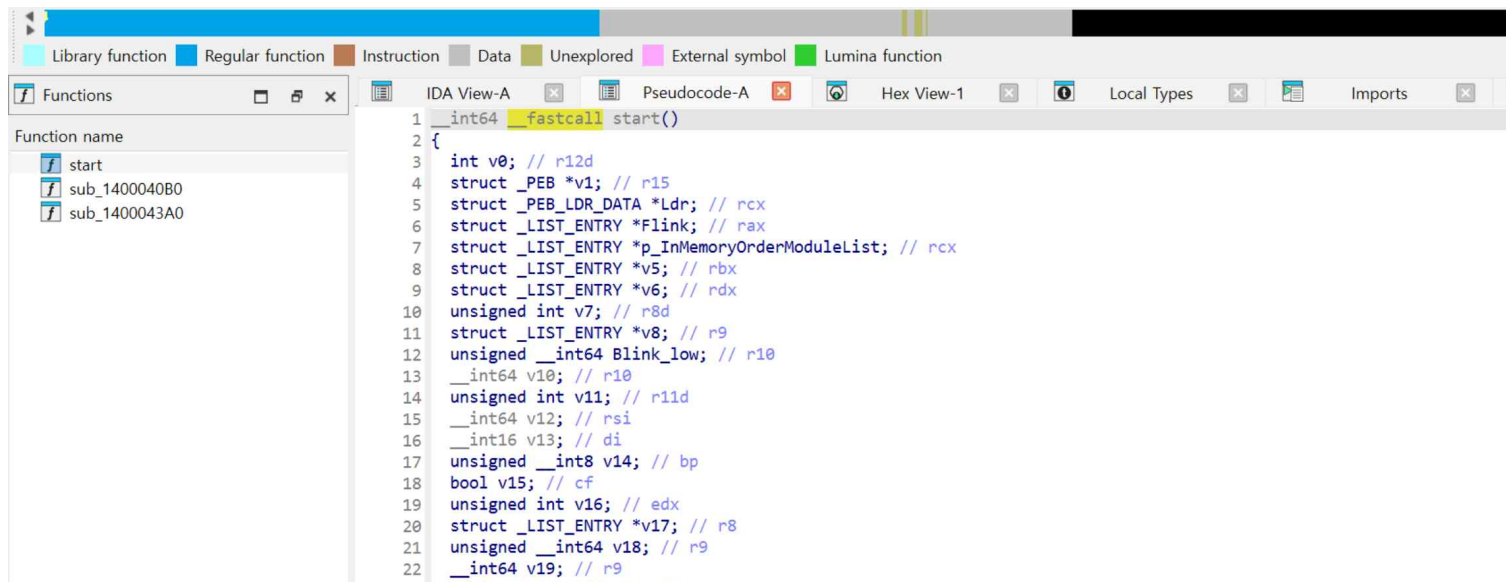
정명훈

우선 프로그램이 복잡하기 때문에 먼저 프로젝트에서 정상 입력으로 준 값을 입력하면

```
C:\>"D:\Downloads\Security_project4_challenge\Security_project4\challenge\reversing_project4_output_fixed.exe"
RVP4 / state-coupled license challenge
Name (3-24 ASCII chars): alice
License: R4-0401C3A4-EFCDAB89-67452301-67165367-7098427E-8973E896
Result: LICENSE ACCEPTED
```

이와 같이 올바른 결과가 나온다. 그 다음에 아래 설명문을 보니 - 디버거, 코드 패치, 하드웨어 중단점, 장시간 단일 스텝 실행은 내부 상태에 영향을 준다. 다고 하는 걸 보니 이번 프로젝트는 기존과는 다르게 상당히 어려울 것이라는 걸 알 수 있다. 하지만 위의 예시를 통해서 license의 형식을 파악했다는 점은 큰 성과이다. 자 그러면 일단 ida에 먼저 이 파일을 올려서 실행시켜 보자

우선 먼저 Section, Entry Point를 분석해보려고 한다. 그러기 위해서는 ida에서 jump -> jump to entry point로 가면



start함수가 사실상 main함수의 기능을 수행하고 있다는 사실을 쉽게 알 수 있다. 그 다음에 section부분을 찾아보면

Line 1 of 3, /start										UNKNOWN 0000000140006041: .data:0000000140006041 (Synchronized with Hex View-1)									
Output										Strings									
										Segments									
Name	Start	End	R	W	X	D	L	Align	Base	Type	Class	AD	es	ss	ds	fs	gs		
.text	0000000140001000	0000000140005000	R	.	X	.	L	para	0001	public	CODE	64	FFFFFF...	FFFFFF...	0003	FFFFFF...	FFFFFF...		
.rdata	0000000140005000	0000000140006000	R	.	.	.	L	para	0002	public	DATA	64	FFFFFF...	FFFFFF...	0003	FFFFFF...	FFFFFF...		
.data	0000000140006000	0000000140007000	R	W	.	.	L	para	0003	public	DATA	64	FFFFFF...	FFFFFF...	0003	FFFFFF...	FFFFFF...		

이렇게 비교적 간단한 text, rdata, data 등 세가지 영역만으로 이루어져 있다는 사실을 파악할 수 있다.

그 다음에는 Import Table을 봤더니 비어있어 왜 비어있는지 알아내기 위해서 우선적으로 사실상 main함수의 기능을 하는 start함수의 디컴파일된 결과를 이용해서 분석을 해보니 변수가 너무 많고 반복문과 qword 등의 문이 많았다. 또한 qword_7FF677E360B0(..., "Result: ", ...);, qword_7FF677E360B8(0);, qword_7FF677E360A8(...); 와 같이 API 이름 대신 전부 함수 포인터로 호출한다는 걸 알 수 있다. 그래서 IDA Imports 창은 비어 있고, Strings 창에도 GetProcAddress, ReadFile, WriteFile 같은 이름이 안 보이는 걸 확인할 수 있었다.

그 다음 목표는 입력 함수와 이름 정규화 루틴 복원이다. 우선 사실상 main함수의 기능을 하는 start함수의 이름을 main으로 바꾸고 분석을 통해 입력함수를 찾아보자

우선 위의 cmd창을 기억해보면 Name (3-24 ASCII chars):이와 같은 설명문이 나오고 이름을 입력받았으므로 이 정보를 이용해서 ida에서 search->text를 누른 다음에 그 쪽 부분으로 가면

```

63 ;
64 }
65 LODWORD(v454) = 0;
66 qword_7FF677E360B0(v272, "RVP4 / state-coupled license challenge\r\nName (3-24 ASCII chars): ", 65, &v454, 0);
67 LODWORD(v454) = 0;
68 if ( (unsigned int)qword_7FF677E360A8(qword_7FF677E36108, byte_7FF677E36000, 159, &v454, 0) )
69 {
70     v273 = (unsigned int)v454;
71     if ( (_DWORD)v454 )
72     {

```

이렇게 입력받는 부분을 쉽게 찾을 수 있다. 그 다음에는 실제 입력과정이 어떻게 이루어지는지를 확인하기 위해서 각각의 입력과 입력을 가공하는 함수들을 자세히 살펴보면

우선 위의 이름 안내 함수는 인자와 양식이

```
function(
```

```
    handle,
```

```
    buffer,
```

```
    length,
```

```
    &bytes_written,
```

```
    overlapped
```

```
);과 같고 이는 BOOL WriteFile(
```

```
    HANDLE        hFile,
```

```
    LPCVOID        lpBuffer,
```

```
    DWORD          nNumberOfBytesToWrite,
```

```
    LPDWORD        lpNumberOfBytesWritten,
```

```
    LPOVERLAPPED lpOverlapped
```

);와 동일합니다 따라서 qword_7FF677E360B0 → pWriteFile로 함수 이름을 바꿀 수 있다

그리고 그 바로 아래의 코드

```
LODWORD(v454) = 0;
```

```
if ( (unsigned int)qword_7FF677E360A8(
```

```
    qword_7FF677E36108,
```

```
    byte_7FF677E36000,
```

```
    159,
```

```
    &v454,
```

0))

를 보면 이는 이름을 입력받는 코드이며 형식은

```
function(  
  
    qword_7FF677E36108,    // 핸들  
  
    byte_7FF677E36000,    // 입력 버퍼  
  
    159,                  // 최대 입력 크기  
  
    &v454,                // 실제 읽은 바이트 수  
  
    0                     // OVERLAPPED = NULL
```

);다음과 같고 이는 아래에 나온 ReadFile의 원형과 정확히 일치한다

```
BOOL ReadFile(  
  
    HANDLE        hFile,  
  
    LPVOID        lpBuffer,  
  
    DWORD         nNumberOfBytesToRead,  
  
    LPDWORD       lpNumberOfBytesRead,  
  
    LPOVERLAPPED lpOverlapped  
  
);
```

qword_7FF677E360A8 → pReadFile

qword_7FF677E36108 → hStdin

byte_7FF677E36000 → rawName

v454 → bytesRead

따라서 다음과 같이 함수의 이름을 복원할 수 있고 복원된 코드는 아래와 같다.

```
DWORD bytesRead = 0;
```

```

if (pReadFile(
    hStdin,
    rawName,
    159,
    &bytesRead,
    NULL))
{
    // 이름 전처리
}

```

그 다음에는 그 앞쪽의 hStdin이 만들어지는 부분을 보면

```

qword_7FF677E36108 =
    ((__int64 (__fastcall *)...))v445.m128i_i64[1])(
        4294967286LL,
        ...
    );

```

```
v272 = qword_7FF677E360A0(4294967285LL);
```

```
qword_7FF677E36110 = v272;
```

이와 같은 코드가 존재하는데 그 코드의 상수를 부호있는 32비트로 보면

4294967286 = 0xFFFFFFFF6 = -10, 4294967285 = 0xFFFFFFFF5 = -11 각각 이런 값을 나타내

고 window의 표준 핸들 상수는 다음 #define STD_INPUT_HANDLE ((DWORD)-10)

, #define STD_OUTPUT_HANDLE ((DWORD)-11)과 같기 때문에 qword_7FF677E360A0 → pGetStdHandle이라는 걸 알 수 있다. 또한 아래의 결과 출력에서 나오는

```

3 LABEL_367:
4   v283 = 0;
5 LABEL_368:
6   LODWORD(bytesRead) = 0;
7   pWriteFile(qword_7FF677E36110, "Invalid name.\r\n", 15, &bytesRead, 0);
8   qword_7FF677E360B8(2);
9 LABEL_369:
0   LODWORD(bytesRead) = 0;
1   pWriteFile(qword_7FF677E36110, "License: ", 9, &bytesRead, 0);
2   LODWORD(bytesRead) = 0;
3   if ( (unsigned int)pReadFile(hStdin, byte_7FF677E36120, 199, &bytesRead, 0) )
4   {
5       v288 = bytesRead;

```

qword_7FF677E360B8 함수는 qword_7FF677E360B8(2);,qword_7FF677E360B8(3);

qword_7FF677E360B8(4);,qword_7FF677E360B8(0);와 같이 호출되기 때문에 ExitProcess라는 걸 알 수 있다.

그 다음에는 IDA가 함수 포인터를 _QWORD 기반으로 잘못 표시하므로 직접 타입을 지정하면 디컴파일 결과가 크게 개선하려고 하는데 우선 pReadFile은 BOOL (__stdcall *pReadFile)(

void *hFile,

void *lpBuffer,

unsigned int nNumberOfBytesToRead,

unsigned int *lpNumberOfBytesRead,

void *lpOverlapped

);과 같은 타입으로 y를 눌러 바꾸고 그 다음에 pWriteFile함수는 BOOL (__stdcall *pWriteFile)(

void *hFile,

const void *lpBuffer,

unsigned int nNumberOfBytesToWrite,

unsigned int *lpNumberOfBytesWritten

```
void *lpOverlapped
```

);다음과 같이 지정하고 pGetStdHandle함수는 void *(__stdcall *pGetStdHandle)(int nStdHandle);와 같이 저장한다.

pExitProcess는 void (__stdcall *pExitProcess)(unsigned int exitCode); 와 같이 반환값과 매개변수값을 변경한다.

그렇게 다 변경한 이후에 ReadFile 다음에 나오는 첫 번째 루프를 분석해 보면

```
v273 = (unsigned int)v454;
```

```
if ((_DWORD)v454)
```

```
{
```

```
while (1)
```

```
{
```

```
    v279 = (unsigned int)(v273 - 1);
```

```
    v280 = (unsigned __int8)byte_7FF677E36000[v279];
```

```
    if (v280 != 13 && v280 != 10)
```

```
        break;
```

```
    --v273;
```

```
    if (!v279)
```

```
        goto LABEL_336;
```

```

    }

    byte_7FF677E36000[v273] = 0;

}

```

이런게 나오는데 이걸 해석하기 위해 10,13의 의미가 무엇인지를 생각해 본다면 13 = 'Wr', 10 = 'Wn'의 의미를 가지기 때문에 이 루프는 따라서 입력 끝의 CR/LF를 반복 제거한다. 그 점을 고려해 이걸 완벽히 c프로그래밍 스타일로 복원한다면 `size_t rawLength = bytesRead;`

```

while (rawLength > 0)
{
    char ch = rawName[rawLength - 1];

    if (ch != 'Wr' && ch != 'Wn')
        break;

    rawLength--;
}

```

`rawName[rawLength] = 'W0';`와 같은 형태이고 `ReadFile`은 `fgets`와 달리 자동으로 널 종료하지 않기 때문에 프로그램이 직접 'W0'을 넣고 있다.

그 다음 코드를 보면

```

v275 = 0;

while (byte_7FF677E36000[v275] == 32)
{
    if (v273 == ++v275)
    {
        ...
    }
}

```

```
}
```

}위와 같은 형태로 코드가 작성되어 있는데 여기서 32는 아스키 코드상 공백 ' '을 의미하고 c스타일로 복원하면 `size_t begin = 0;`

```
while (begin < rawLength && rawName[begin] == ' ')
```

`begin++;` 과 같은 형태로 나오는데 주의할 점은 `w`t가 아닌 일반 공백 ' '만 지운다는 것이다.

그 아래 코드

```
v276 = (unsigned int)v275;
```

```
v277 = v273 + 1;
```

```
while (v273 > v276)
```

```
{
```

```
--v277;
```

```
if (byte_7FF677E36000[--v273] != 32)
```

```
goto LABEL_352;
```

```
}
```

```
v277 = v274;
```

도 마찬가지로 공백을 지우는데 여기서는 뒤쪽의 공백을 지우고 있다.c코드 형태로 작성시

```
size_t end = rawLength;
```

```
while (end > begin && rawName[end - 1] == ' ')
```

```
end--;
```

다음과 같은 의미를 가지고 이런 전처리들을 거치게 되면 입력 " Alice Bob "은 처리 범위가 "Alice Bob"으로 좁혀진다.

그 다음에 ida 정규화 표현에

```
v284 = byte_7FF677E36000[v276];
```

```
if ((unsigned __int8)(v284 - 127) < 0xA1u)
```

```
    goto LABEL_367;
```

와 같은 조건이 나오는데 ida표현만으로는 이해가 힘들지만 이를 8비트 unsigned 연산으로 풀면 허용 범위는 0x20 ~ 0x7E로 출력 가능한 아스키 코드만 허용한다.

그 다음에 그 아래 부분의

```
if (v284 != 32)
```

```
    break;
```

```
v285 = 1;
```

```
if (!v282)
```

```
    goto LABEL_359;
```

```
v282 = 1;
```

```
goto LABEL_362;
```

```
byte_7FF677E36250[v287] = v286;
```

```
v282 = v285;
```

코드에서 v282는 직전 출력 문자가 공백이었는지를 나타내고 이는 복원하면

```
bool previousWasSpace = false;
```

```
if (ch == ' ')
```

```
{
```

```
    if (previousWasSpace)
```

```

        continue;

        previousWasSpace = true;
    }

    else{    previousWasSpace = false;}

```

다음과 같으며 연속 공백을 공백 하나로 축소하는 기능을 한다. 예를 들면 다음과 같은 결과를 가져온다. 입력: "Alice Bob"에 대해서 출력: "alice bob"이 나온다.

그 다음 코드

```

v18 = (unsigned __int8)(v284 - 65) < 0x1Au;

v286 = v284 + 32;

if (!v18)

    v286 = byte_7FF677E36000[v276];

```

를 해석하려면 65는 아스키의 문자 'A'를 나타내고 0x1A는 알파벳 개수 26라는 걸 먼저 파악한 다음에 c코드로 바꿔서 보면

```

if (ch >= 'A' && ch <= 'Z')

    ch += 'a' - 'A';

```

와 같은 의미를 지닌 코드로 모든 대문자나 소문자를 전부 소문자로 바꾸는 코드의 기능을 수행한다.

그 다음에 코드를 보면 정규화된 문자는 다음 버퍼에 `byte_7FF677E36250[v287] = v286;` 와 같이 저장된다는 걸 확인할 수 있다.

`byte_7FF677E36000` → `rawName`, `byte_7FF677E36250` → `normalizedName`, `v276` → `inputIndex`, `v283` → `normalizedLength`, `v282` → `previousWasSpace` 따라서 다음과 같이 이름을 바꿔 가독성을 올릴 수 있다.

그 다음의 조건을 보면

```
if (++v276 >= v281 || v283 >= 0x1F)
```

```
{
```

```
    byte_7FF677E36250[v283] = 0;
```

```
    ...
```

}와 같이 길이를 제한하는 코드가 나와있는데 이걸 c코드로 변환하면

```
if (normalizedLength >= 31)
```

```
    stop;
```

다음과 같은 의미를 지닌 코드가 나오고 이는 정규화 버퍼 자체는 31글자까지 작성가능하다는 사실을 알 수 있다.

그리고 그 다음에 코드를 보면 ida기준으로

```
if (v283 - 25 <= 0xFFFFFFE9)
```

```
    goto LABEL_368;
```

와 같이 나와있는데 이 조건은 unsigned 정수의 오버플로 때문에 직관적이지 않아 값을 실제 대입해보면 3-24사이의 길이의 문자열을 요구한다는 사실을 알 수 있다. 즉 원래 코드는

```
if (normalizedLength < 3 || normalizedLength > 24)
```

```
    invalid;
```

와 같은 의도를 가진 코드이다. 이는 Name (3-24 ASCII chars):의 프록포트 입력 창의 조건과 동일한데 중요한 점은 이 3-24조건이 정규화를 거친 문자열 기준이라는 것이다.

예를 들면 원본 입력: "A B", 정규화: "a b",길이:3,결과:유효 하다는 걸 알 수 있다

이제 이전의 입력과정을 한 번에 c코드 스타일로 정리하면

```
char rawName[160];
```

```
char normalizedName[32];
```

```
DWORD bytesRead = 0;
```

```
WriteFile(
```

```
    hStdout,
```

```
    "RVP4 / state-coupled license challenge\r\n"
```

```
    "Name (3-24 ASCII chars): ",
```

```
    65,
```

```
    &written,
```

```
    NULL
```

```
);
```

```
if (!ReadFile(
```

```
    hStdin,
```

```
    rawName,
```

```
    159,
```

```
    &bytesRead,
```

```
    NULL))
```

```
{

    bytesRead = 0;

    rawName[0] = '\0';

}

size_t normalizedLength;

if (!normalize_name(

    rawName,

    bytesRead,

    normalizedName,

    sizeof(normalizedName),

    &normalizedLength))

{

    WriteFile(

        hStdout,

        "Invalid name.\r\n",

        15,

        &written,

        NULL

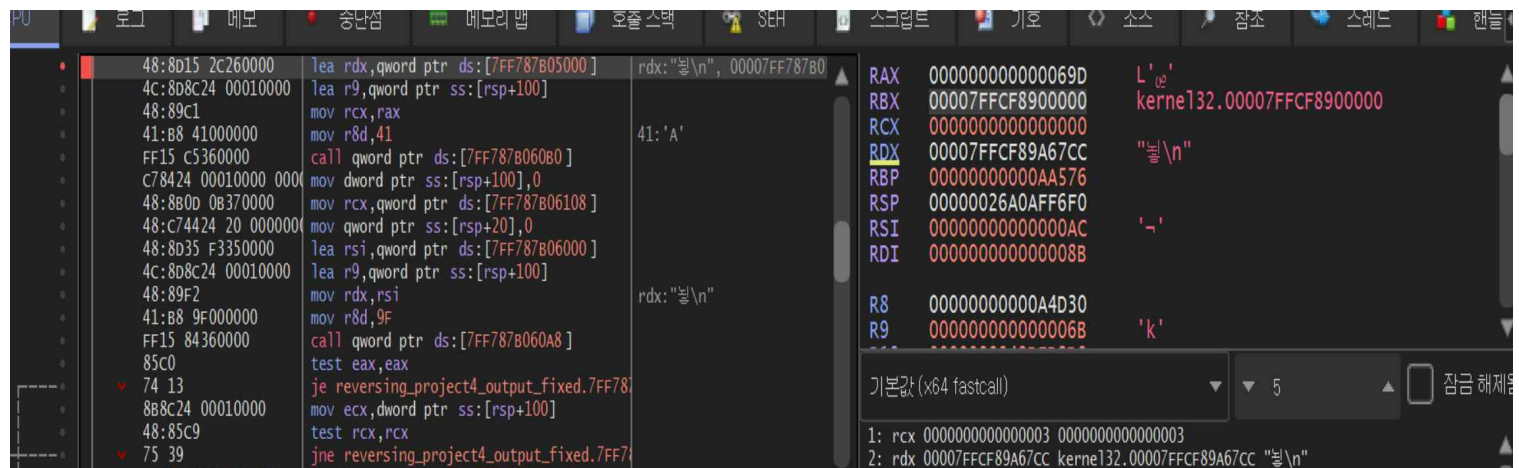
    );
```

```
ExitProcess(2);

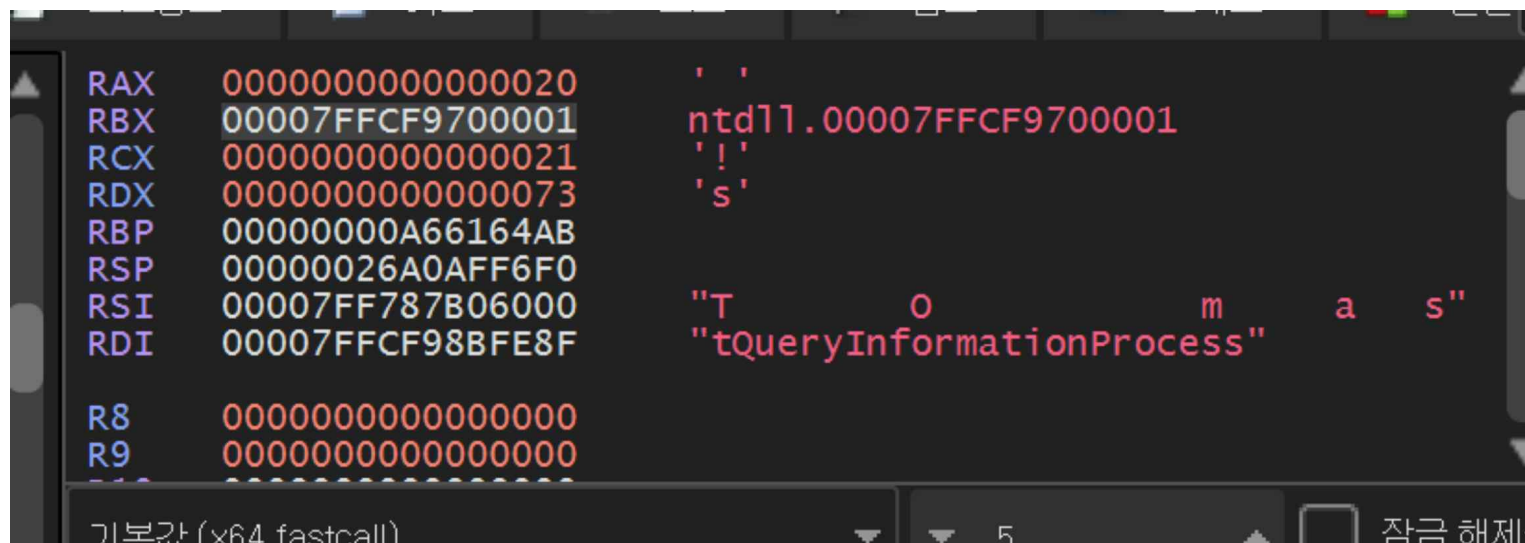
}
```

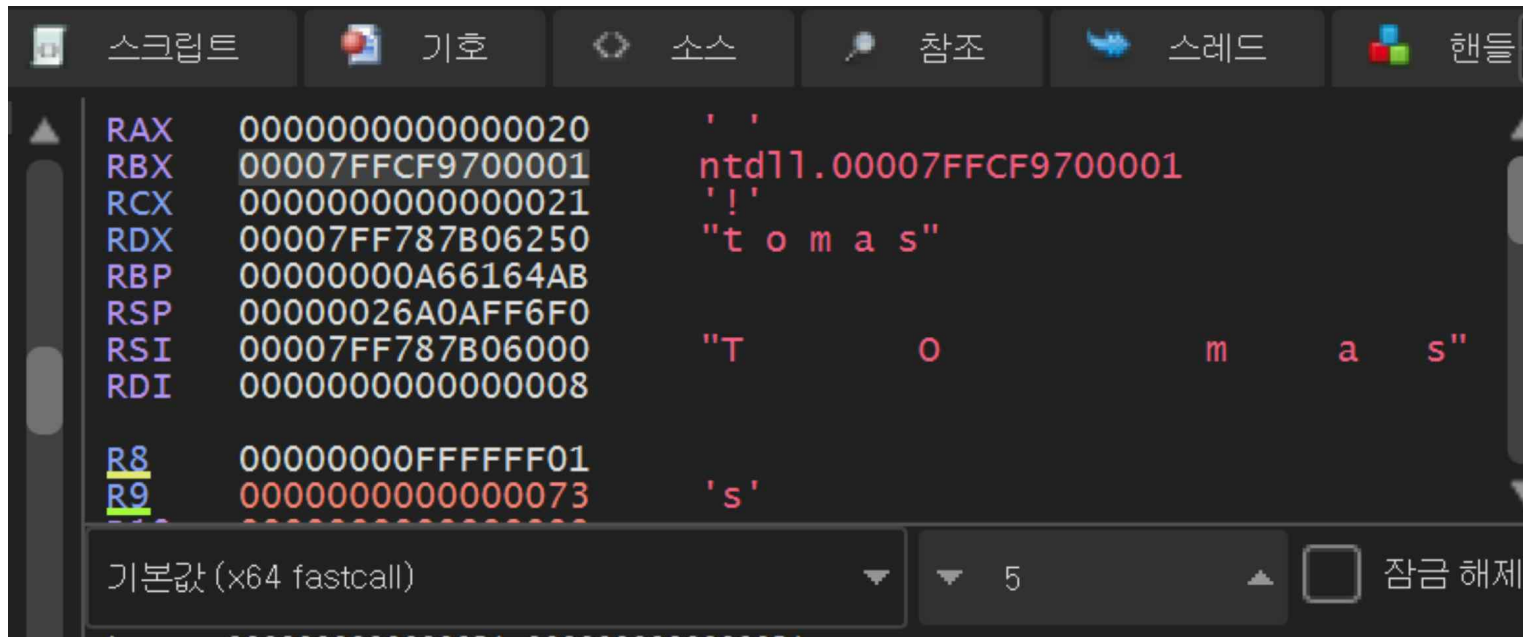
다음과 같다는 걸 확인할 수 있다.

그 다음에 동적분석으로 정적분석한 결과가 맞는지를 확인해 보기 위해 xdbg64에 들어가고 문자열 찾기에서 입력 부분의 안내 문자열을 찾으면 아래와 같이 나온다.



그 다음에 ida와 xdbg의 주소를 맞추고 ida를 이용해서 각각 정적 함수의 입력 정규화 및 검증 과정하는 지점에 중단점을 설정하고 프로그램을 디버그로 단계별로 실행시키고 레지스터값의 변화를 보면





와 같이 좌우 공백 삭제 중복 공백을 단일 공백으로 변환 등의 기능이 수행됨을 확인할 수 있었다.

자 이제 라이선스 24바이트 필드 분리를 한 번 하도록 하자 그러기 위해서는 먼저 라이선스를 입력받는 부분에 가서 이 부분에서 입력받은 값을 어떻게 활용하는지를 파악해야 한다.

```

93  v447 = 0;
94  if ( v289 < 4 || Buffer[0] != 82 || byte_7FF787B06121 != 52 || byte_7FF787B06122 != 45 )
95  {
96  LABEL_395:
97      LODWORD(bytesRead) = 0;
98      pWriteFile(
99          hFile,
100         "Invalid format. Expected R4-XXXXXXXX-XXXXXXXX-XXXXXXXX-XXXXXXXX-XXXXXXXX-XXXXXXXX\r\n",
101         0x53u,
102         (unsigned int *)&bytesRead,
103         nullptr);
104      pExitProcess(3u);

```

우선 입력받은 부분을 가면 형식은 파악이 된다 그 다음에 파싱 부분을 보면

```
if ( (v296 & 1) != 0 )
```

```
*((_BYTE *)&v446 + (v296 >> 1)) |= v300;
```

```
else
```

```
*((_BYTE *)&v446 + (v296 >> 1)) = 16 * v300;
```

이 부분이 핵심인데 이 부분의 의미는 짝수 번째 16진수 문자는 상위 4비트, 홀수 번째 문자는 하위 4비트에 넣고 있어. 따라서 v446 주소부터 24바이트가 연속 저장된다는 걸 알 수 있다.

그 다음 파싱이 끝난 이후에 묶이는 걸 보면 우선 첫 번째 필드는

```
v441.m128i_i32[0] = (unsigned __int16)v446; // byte 0~1
```

```
v350 = v441.m128i_i32[2] << 16; // byte 2~3
```

```
v357 = v441.m128i_i32[0] + v350;
```

다음과 같이 묶일 수가 있고 이를 쉽게 표현하면

```
field0 = license[0]
```

```
| license[1] << 8
```

```
| license[2] << 16
```

```
| license[3] << 24;
```

다음과 같아 결국 DWORD 하나다.

그 다음에 두 번째 필드를 보면

```
v323 = BYTE4(v446) | BYTE5(v446) << 8;
```

```
v335 = BYTE6(v446) << 16 | v323;
```

```
v341 = BYTE7(v446) << 24 | v335;
```

이렇게 먼저 byte4-7을 만들고 그 다음에

```
v342 = (_QWORD)WORD4(v446) << 32; // byte 8~9
```

```
v352 = WORD5(v446) << 48; // byte 10~11
```

```
v351 = v341 | v342;
```

```
v358 = HIDWORD(v351) | HIDWORD(v352);
```

와 같이 byte 8~11을 상위 부분에 붙인다.

ida디컴파일이 매우 더럽게 나왔지만 이는 실질적으로는 `field1 = *(uint64_t *)&license[4];` 다음과 같이 보는게 자연스럽다.

똑같은 방법으로 세 번째 필드를 보면 역시 마찬가지로 방법으로

byte 12~15+byte 16~17+byte 18~19를 결합하고 있다.

```
관련값은 v443 = BYTE12(v446);
```

```
LODWORD(v439) = BYTE13(v446);
```

```
LODWORD(v442) = BYTE14(v446);
```

```
LODWORD(v448) = HIBYTE(v446); // byte 15
```

```
v427 = (unsigned __int16)v447; // byte 16~17
```

```
v445.m128i_i64[1] = WORD1(v447); // byte 18~19
```

와 같으며 이는 `field2 = *(uint64_t *)&license[12];`과 같은 의미를 지닐 거라고 추정할 수 있다.

그리고 역시 마찬가지로 네 번째 필드 역시 `field3 = *(uint32_t *)&license[20];`

라는 것을 알 수 있다.

따라서 전체적인 예상구조는

```
#pragma pack(push, 1)
```

```
typedef struct LicenseFields
```

```
{
```

```

uint32_t field0; // offset 0x00

uint64_t field1; // offset 0x04

uint64_t field2; // offset 0x0C

uint32_t field3; // offset 0x14

} LicenseFields;

#pragma pack(pop)

```

다음과 같으며 합계는 $4 + 8 + 8 + 4 = 24$ 바이트와 같다. 하지만 이를 통해 알 수 있는 건 크기와 오프셋일 뿐이고 각 필드의 의미는 아직 확정할 수 없다. 따라서 지금 단계에서는 편의상

v446 → licensePart0_15 또는 licenseBytes

v447 → licensePart16_23

v357 → licenseField0_u32

v351 → license_field1_partial64

v358 → licenseField1_high32

v353 → licenseField2_partial32 또는 licenseField2_highPartial

v359 → licenseField2_high32

v327 → licenseField3_u32

v301 → normalizedNameHash

v321 → textSectionHash

v330 → environmentFlags 이렇게 이름만 바꿔 놓는다.

이 부분은 내용이 너무 길어서 PEB 기반 module traversal과 export resolver 분석.pdf파일에 내용을 정리해 놔다.

그 다음에는 어떻게 보면 가장 핵심적인 작업이자 정답을 찾아내는 작업인 API 해시 계산기를 python file을 이용해서 제작하는 것이다. 그러기 위해선 가장 먼저

- ① Export 이름 문자열을 가져오는 위치
- ② 문자열을 한 글자씩 처리하는 반복문
- ③ 계산된 값과 목표 해시를 비교하는 위치

이 세 곳부터 ida와 xdbg64에서 이전 단계에서 알아낸 resolveExportRva함수 즉 해시 연산을 거치는 함수에서 알아보자

```
Blink_high = HIWORD(a1[3].Blink);
a3 = *(unsigned int *)((char *)&a1[8].Blink + Blink_high);
if ( *(_DWORD *)((char *)&a1[8].Blink + Blink_high) && *(_DWORD *)((char *)&a1[1].Blink + a3) )
{
    a5 += 2;
    a4 = *(_DWORD *)((char *)&a1[8].Blink + Blink_high + 4);
    v34 = 0;
    while ( 1 )
    {
        v36 = *(unsigned int *)((char *)&a1->Flink + 4 * v34 + *(unsigned int *)((char *)&a1[2].Flink + a3));
        v37 = *((_BYTE *)&a1->Flink + v36);
        if ( v37 )
        {
            v38 = (char *)a1 + v36;
            v39 = 1;
            v35 = 0;
            while ( v37 == v26[v35] )
            {
                v35 = v39;
                v37 = v26[v35];
            }
        }
    }
}
```

0003738 resolveExportRva:120 (7FF787B04338)

위의 스크린샷을 보면 v34 → Export 이름 인덱스, v36 → AddressOfNames[v34]에서 꺼낸 이름 RVA이고, v38 = (char *)a1 + v36 → 모듈 베이스 + 이름 RVA로 만든 실제 API 이름 문자열 포인터이며, v37 → 현재 비교 중인 API 이름 문자라는 걸 알 수 있다. 또한 스크린샷에 나와있는 while루프가 API 함수 이름을 직접 문자열로 비교하는 구조라는 걸 알 수 있다.

그 다음에 계산된 값과 목표 해시를 비교하는 위치를 찾아보면

```

82      do
83      {
84          v22 = *((unsigned __int8 *)&v17->Flink + 2 * v21);
85          if ( v22 == 46 )
86              break;
87          v23 = v22 + 32;
88          if ( (unsigned __int8)(v22 - 65) >= 0x1Au )
89              v23 = *((_BYTE *)&v17->Flink + 2 * v21);
90          v19 = __ROL4__(16777619 * (v23 ^ v19), 5) ^ 0x9E3779B9;
91          ++v21;
92      }
93      while ( v20 != v21 );
94  }
95  if ( v19 == v8 )
96      break;
97  }
98  Flink = Flink->Flink;
99  if ( Flink == p_InMemoryOrderModuleList )
100      return 0;
101  result = 0;
102  if ( v16++ > 0x3F )

```

여기서 `v19 == v8` 부분이 모듈함수를 비교하는 부분이라는 걸 알 수 있다 이제 위의 해시값으로 암호화하는 과정과 그 이전에 IDA에서 발견한 API 해시 상수 → 실제 Windows API 이름 복원해 출력하는 코드를 작성해서 `rvp4_api_hash_resolver_corrected.py`와 `rvp4_module_hash_resolver.py`로 저장했으니 이를 참조하면 된다.

그 다음에는 PEB 디버그 필드 분석해 보자. 우선 디버그 필드를 검사하는 부분을 먼저 찾으면

`textSectionHash` 계산 직후, `environmentFlags`를 초기화하는 부분으로 ida기준 1986번째 줄부터 시작한다. 그 부분의 코드는

```
v329 = (__int64)v431;
```

```
environmentFlags = (v431->BeingDebugged != 0) + 2LL;
```

```
if ( (v431->NtGlobalFlag & 0x70) == 0 )
```

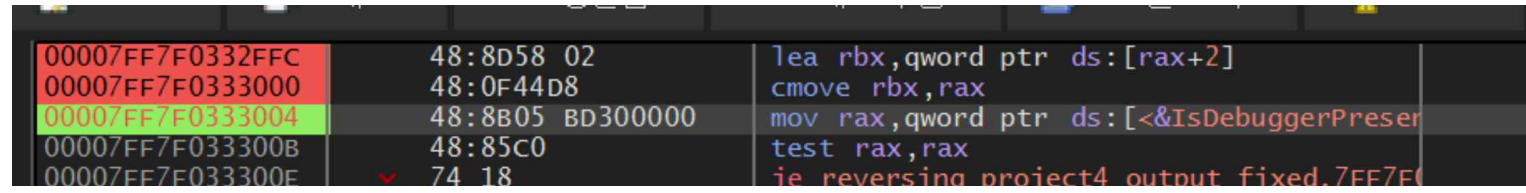
```
    environmentFlags = v431->BeingDebugged != 0;
```

이 부분이고 여기서 PEB 디버그 필드 접근은 `v431->BeingDebugged`와 `v431->NtGlobalFlag` 두 곳이다.

이제 접근 부분을 알았으니 if문 전후 레지스터 값들의 변화를 알기 위해서 동적 분석을 해 보자

```
1986 | environmentFlags = (v431->BeingDebugged != 0) + 2LL;
1987 | if ( (v431->NtGlobalFlag & 0x70) == 0 )
1988 |     environmentFlags = v431->BeingDebugged != 0;
```

우선 이 부분에 breakpoint를 xdbg64에



이렇게 걸고 디버거를 f9를 누르면서

RVP4 / state-coupled license challenge

Name (3-24 ASCII chars): Tomas

License: R4-0401C3A4-EFCDAB89-67452301-67165367-7098427E-8973E896

아래와 같은 입력을 주고 실행하며 레지스터 값들을 xdbg64에서 하나씩 확인해 보면

검사	동적 결과	비트
PEB.BeingDebugged	탐지됨	0x01
PEB.NtGlobalFlag & 0x70	탐지됨	0x02
IsDebuggerPresent()	탐지됨	0x04

이렇게 xdbg64를 사용한 부분이 탐지되었다는 사실을 확인할 수 있다. 그렇지만 뒤의

```

1998     environmentFlags += 4;
1999 }
2000 v334 = normalizedNameHash ^ (normalizedNameHash >> 29);
2001 v335 = v434.m128i_i64[1] + v323;
2002 v336 = v434.m128i_i64[0] + v325;
2003 NumberOfBytesWritten = 0;
2004 v337 = qword_7FF7F03360C0(v329, v328);
2005 if ( qword_7FF7F03360D0 )
2006 {
2007     v338 = qword_7FF7F03360D0(v337, &NumberOfBytesWritten);
2008     v339 = environmentFlags + 8;
2009     if ( !NumberOfBytesWritten )
2010         v339 = environmentFlags;
2011     if ( v338 )
2012         environmentFlags = v339;
2013 }
2014 v340 = 0x9E3779B185EBCA87uLL * v334;
2015 v341 = v324 | v335;
2016 v342 = (_QWORD)Flink << 32;
2017 v343 = HIDWORD(v326) | HIDWORD(v336);
2018 savedNtHeadersOffsetc = savedNtHeadersOffsetb << 32;

```

이 부분에도 environmentFlag가 다음과 같이 존재하기 때문에

0x001 PEB.BeingDebugged

0x002 PEB.NtGlobalFlag

0x004 IsDebuggerPresent 계열

0x008 CheckRemoteDebuggerPresent 계열

0x010 NtQueryInformationProcess class 7

0x020 NtQueryInformationProcess class 31

0x040 NtQueryInformationProcess class 30

0x080 예외 컨텍스트/디버그 레지스터 검사

0x100 시간 검사

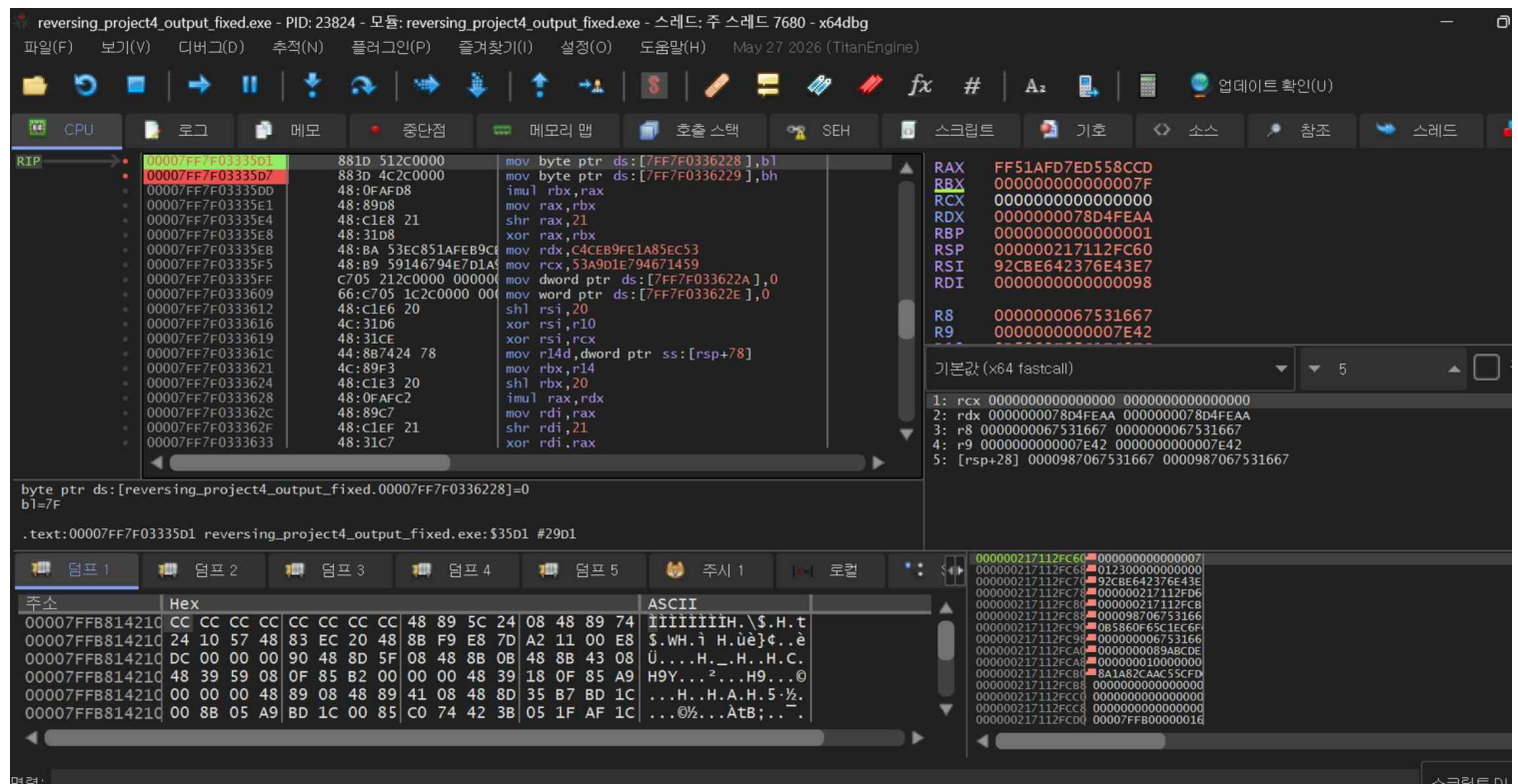
단순 패치로 해결하기는 까다롭다. 또한 $v463 = \text{textSectionHash} \wedge 0x376E43E7$;와 같이 text값을 해시하기 때문에 패치를 진행하기도 까다롭다. 이를 해결하기 위해서 최종 상태 결합 지점 하나만 패치하는 것으로 environmentFlags 최종값과 textSectionHash 최종값을 vm함수에 들어가기 전에 올바른 값으로 설정하는 것이다.

```

2147 WORD4(xmmword_7FF7F0336220) = environmentFlags;
2148 *(_DWORD *)((char *)&xmmword_7FF7F0336220 + 10) = 0;
2149 HIWORD(xmmword_7FF7F0336220) = 0;
2150 v364 = savedPeb ^ 0x64C7920094671459LL;
2151 v365 = v440.m128i_u32[0];

```

즉 여기 부분 직전에 값을 올바르게 설정해야 한다. 그러기 위해서 2147번째 줄에 xdbg64를 이용해서 중단점을 걸고



나의 환경변수값을 보면 0x7f가 나오고 이것 0으로 만들어야 하기 때문에 기존 코드 부분의

```
mov byte ptr ds:[7FF7F0336228],b1
```

```
mov byte ptr ds:[7FF7F0336229],bh
```

```
7FF7F03335D1 xor ebx, ebx
```

7FF7F03335D3 mov word ptr [7FF7F0336228], bx 이렇게 바꾼다. 그렇게 하면 환경변수의 값은 정상값인 0이 항상 나온다. 하지만 여기서 끝나면 안된다 이렇게

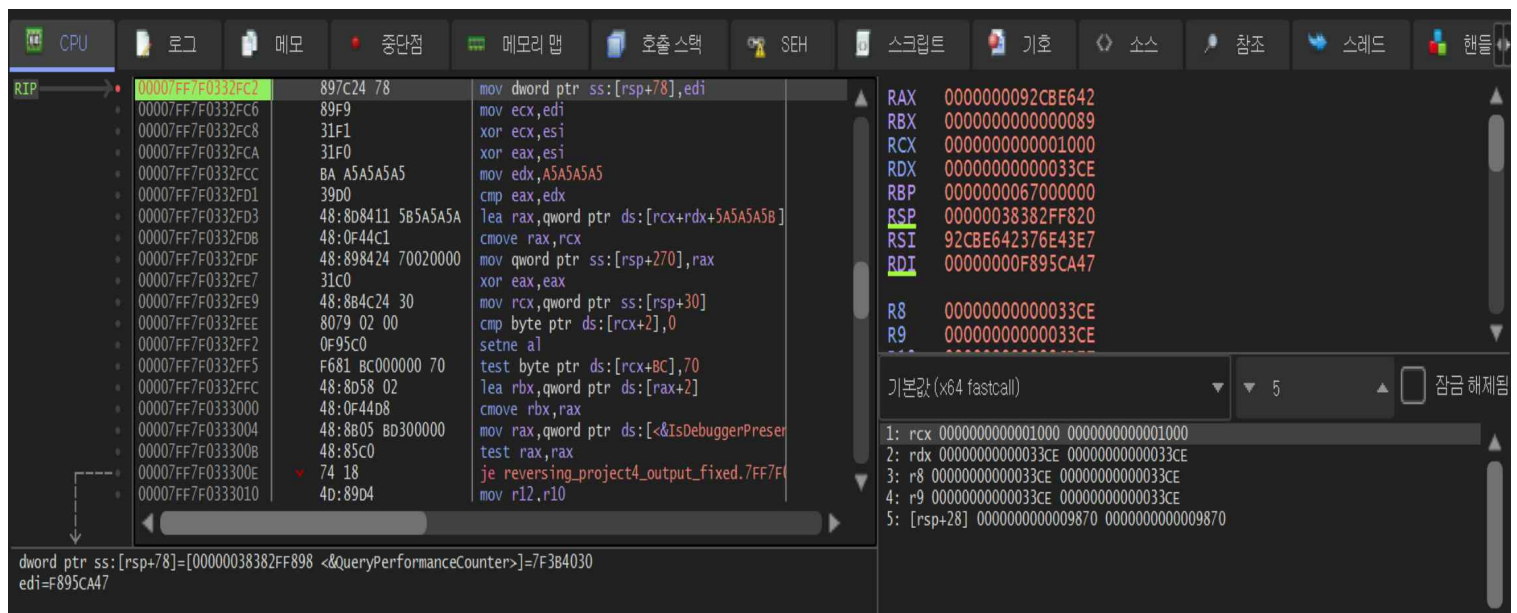
른 값으로 바꿔줄 필요가 있다. 그러기 위해서는 기존 exe파일에서 올바른 text영역의 해시값부터 먼저 구할 필요가 있는데 우선 해시값을 구하는 부분을 ida를 통해서 보면

```

1981 11CENSEL 1E103_024 = 11CWORD(11CENSEL 1E103_024);
1982 v440.m128i_i32[0] = textSectionHash;
1983 v328 = 2779096485LL;
1984 v463 = textSectionHash ^ 0x376E43E7;
1985 v329 = (__int64)v431;
1986 environmentFlags = (v431->BeingDebugged != 0) + 2LL;
1987 if ( (v431->NtGlobalFlag & 0x70) == 0 )
1988     environmentFlags = v431->BeingDebugged != 0;
1989 if ( qword_7FF7A0C160C8 )
1990 {
1991     v331 = v323;
1992     v332 = v325;
1993     v333 = qword_7FF7A0C160C8(v431, 2779096485LL);

```

이렇게 나와있고 여기서 1982번째 줄에서 레지스터를 확인하면 올바른 해시값을 구할 수 있다.



여기서 rdi의 레지스터값을 보면 0x00000000F895CA47이고 따라서 올바른 해시값인 edi는 0xF895CA47라는 걸 알 수 있다. 자 이제 이렇게 얻은 올바른 해시값을 패치된 실행파일에 넣어보려고 했지만 빈 공간을 찾아 영구 패치하기가 너무 어려워서 한 번 이 해시값이 어디에 쓰이는 지 알아본 결과 이 해시값은 이름을 serial로 변환하는데 사용이 되지만 기존과 달라진다고 애러가 생기지는 않기 때문에 그냥 바뀐 상태로 vm을 나중에 파이썬 코드로 복원하는 것이 낫다는 판단을 내렸다.

따라서 안티디버깅을 방지하기 위한 패치만 진행했다

이제 이렇게 안티디버깅 코드까지 우회했으니 다음으로 할 것은 바로 VM 복원이다. 이를 위해서 먼저 opcode permutation 생성 알고리즘 복원을 해야한다
그러기 위해서는 먼저

```

9  {
0  v311 = v309 + *((unsigned __int16 *)((char *)ImageBaseAddress + v309 + 20));
1  v312 = (__int64)ImageBaseAddress + v311 + 24;
2  v313 = 40 * v310;
3  v314 = 0;
4  while ( *(_BYTE *)(v312 + v314) != 46
5          || *(_BYTE *)(v312 + v314 + 1) != 116
6          || *(_BYTE *)(v312 + v314 + 2) != 101
7          || *(_BYTE *)(v312 + v314 + 3) != 120
8          || *(_BYTE *)(v312 + v314 + 4) != 116 )
9  {
0  v314 += 40;
1  if ( v313 == (_DWORD)v314 )
2  goto LABEL_414;
3  }

```

이 부분에 가면 이는 2E 74 65 78 74는 각각 . t e x t에 대응되기 때문에 이는 .text section의 해더를 찾는 코드라는 걸 확인할 수 있다.

```

9  {
10 v314 += 40;
11 if ( v313 == (_DWORD)v314 )
12 goto LABEL_414;
13 }
14 v315 = (char *)ImageBaseAddress + v311;
15 v316 = *(_DWORD *)&v315[v314 + 32];
16 if ( v316 )
17 {
18 v317 = *(_DWORD *)&v315[v314 + 36];
19 if ( v316 == 1 )
20 {
21 textSectionHash = -1585668547;
22 LODWORD(processedTextBytes) = 0;
23 }

```

그 후에는 그 아래 코드에 각 변수의 용도에 따라서 이름을 붙혀주면 v316 → textVirtualSize , v317 → textVirtualAddress , v314 → sectionHeaderOffset가 된다

그 이후에는 패치된 실행파일의 text영역의 해시 알고리즘을 복원하기 위해서

해시 루프를 보면

```
v318 = -1585668547;
```

```
v319 = 0;
```

```
v407 = v317;
```

```
do
```

```
{
```

```
    v408 = v318 ^ *((unsigned __int8 *)ImageBaseAddress + v407);
```

```
    v409 = (((unsigned int)(73244475 * v408) >> 25)
```

```
        + 785358208 * v408
```

```
        + 2135587861)
```

```
    ^ *((unsigned __int8 *)ImageBaseAddress + v407 + 1);
```

```
    v318 = 785358208 * v409
```

```
        + ((73244475 * v409) >> 25)
```

```
        + 2135587861;
```

```
    v319 += 2;
```

```
    v407 += 2;
```

```
}
```

```
while ( (v316 & 0xFFFFFFFFFE) != v319 );
```

또한 홀수 길이 처리도

```

if ( (v316 & 1) != 0 )
{
    v410 = *((unsigned __int8 *)ImageBaseAddress
           + (unsigned int)v319
           + v317) ^ v318;

    v318 = 785358208 * v410
           + ((unsigned int)(73244475 * v410) >> 25)
           + 2135587861;
}

```

와 같이 포함해야 한다.

이러한 해시루프에서 각 변수의 의미에 따라 변수의 이름을

v318 → textSectionHash, v319 → processedTextBytes, v407 → currentTextRva

v408 → firstByteMixed, v409 → secondByteMixed과 같이 바꾼다. 그 다음에는 이 해시루프의 과정을 파이썬으로 구현하고 또한 xdbg64에서 해시 루프 종료 후에 edi혹은 v318값을 복원해야 한다. 파이썬 파일은 따로 hashfunc.py로 구현했고 xdbg64에서 v318의 값을 확인해 보기 위해 ida와 xdbg의 주소를 맞추고

```

.text:00007FF6A8052FBE      shr     rax, 20h
.text:00007FF6A8052FC2      mov     dword ptr [rsp+2D8h+var_260], edi
.text:00007FF6A8052FC6      mov     ecx, edi
.text:00007FF6A8052FC8      xor     ecx, esi

```

이 부분의 중단점을 설정하고 edi레지스터를 확인해 보면

00007FF6A8052FC2	897C24 78	mov dword ptr ss:[rsp+78],edi
00007FF6A8052FC6	89F9	mov ecx,edi
00007FF6A8052FC8	31F1	xor ecx,esi

RAX	0000000000092CBE642
RBX	0000000000000000089
RCX	00000000000000001000
RDX	0000000000000000033CE
RBP	00000000000670000000
RSP	0000000075D19DFA90
RSI	92CBE642376E43E7
RDI	0000000000EF424767
R8	00000000000000033CE

다음과 같이 0xEF424767라는 걸 알 수 있고 따라서 이를 통해서 text의 해시값이 0xEF424767라는 걸 알 수 있다.

하지만 놀랍게도 이는 잘못된 값이다 왜냐하면 지금 중단점이 2개나 걸려있고 전부 다 소프트웨어 중단점이기 때문에 이는 해당 명령어의 첫 바이트를 임시로 0xCC(INT3)로 바꿔서 멈추게 하기 때문에 text의 해시값이 이상한 값이 나온다. 이를 해결하기 위해서 기존의 중단점을 모두 해제하고

RIP →	00007FF6A8052FC2	897C24 78	mov dword ptr ss:[rsp+78],edi
	00007FF6A8052FC6	89F9	mov ecx,edi

이 부분에만 하드웨어 중단점을 설정하고 다시 레지스터 edi값을 확인해 보면

RSI	92CBE642376E43E7
RDI	000000000019F8806B
R8	00000000000000033CE
R9	00000000000000033CE
R10	000000000000000CDEF
R11	0000000000000001667

위와 같이 edi 즉 text의 최종해시값이 0x19F8806B임을 알 수 있고 이는 파이썬으로 해시함수를 구현해

```
PS D:\Downloads\Security_project4_challenge\Security_project4\challenge>
python hashfunc.py reversing_project4_output_fixed_v2.exe
```

파일 : reversing_project4_output_fixed_v2.exe

.text VirtualAddress : 0x00001000

.text VirtualSize : 0x000033CE

.text RawOffset : 0x00000400

.text RawSize : 0x00003400

초기 해시 : 0xA17C9E3D

최종 textSectionHash : 0x19F8806B

PS D:\Downloads\Security_project4_challenge\Security_project4\challenge>

다음같이 실행해서 얻은 값과 같다.

이를 통해서 중단점 설정시에도 소프트웨어적인 방법을 선택시 텍스트 영역이 바뀔 수 있음을 알게 되었다.

이렇게 해시값을 얻었음으로 해시값을 저장하는 위치를 ida에서 찾아보면

```

1953 v324 = HIDWORD(v434);
1954 v427.m128i_i32[0] = textSectionHash;
1955 v325 = 2779096485LL;
1956 v450 = textSectionHash ^ 0x376E43E7;
1957 v326 = (__int64)v418;

```

위와 같고 나중에 이 값들이 permutation에서

```
v354 = v427.m128i_u32[0];
```

```
v355 = v419 ^ ((unsigned __int64)v427.m128i_u32[0] << 32);
```

이런 식으로 활용되기 때문에

```
v354 = textSectionHash
```

```
v355 = v419 XOR (textSectionHash << 32) 라는 걸 알 수 있다.
```

그 다음에 위의 v355를 만드는 데 쓰이는 v419값을 추적해 보면

$v419 = v334 \wedge \text{HIDWORD}(v334)$;이고

$v334 = 0x9E3779B185EBCA87uLL * v330$;이며

$v330 = v298 \wedge (v298 \gg 29)$;라는 걸 알 수 있다.

```

1849 {
1850     v302 = 0x100000001B3LL * (v298 ^ (unsigned __int8)byte_7FF6A8056250[v300]);
1851     v303 = ((0x80000000D9800000uLL * (v298 ^ (unsigned __int8)byte_7FF6A8056250[v300])) | (v302 >> 41))
1852         ^ v302
1853         ^ (unsigned __int8)byte_7FF6A8056250[v300 + 1];
1854     v298 = (0x100000001B3LL * v303) ^ ((0x80000000D9800000uLL * v303) | ((0x100000001B3LL * v303) >> 41));
1855     v300 += 2;
1856 }
1857 while ( v301 != v300 );
1858 }
1859 if ( (v299 & 1) != 0 )
1860     v298 = (0x100000001B3LL * ((unsigned __int8)byte_7FF6A8056250[v300] ^ v298))
1861         ^ ((0x80000000D9800000uLL * ((unsigned __int8)byte_7FF6A8056250[v300] ^ v298))
1862         | ((0x100000001B3LL * ((unsigned __int8)byte_7FF6A8056250[v300] ^ v298)) >> 41));
1863 }
1864 v304 = BYTE4(v433);

```

v298은 위의 스크린샷에 나와 있듯이 정규화된 이름 버퍼인 byte_140006250의 각 값을 이용해서 갱신된다. 따라서 v419는 입력한 이름의 정규화된 형태라는 걸 알 수 있다. 이에 맞춰서 각 변수의 이름을 $v298 \rightarrow \text{normalizedNameHash64}$

$v330 \rightarrow \text{nameHashFolded}$, $v334 \rightarrow \text{nameHashProduct}$, $v419 \rightarrow \text{permutationNameState}$ 로 바꾼다.

이를 위해서 opcode를 생성하는 지점에 찾아가면(1970,2099)

```

2097 v354 = v427.m128i_u32[0];
2098 v355 = v419 ^ ((unsigned __int64)v427.m128i_u32[0] << 32);
2099 si128 = _mm_load_si128((const __m128i *)&xmmword_140005200);
2100 v436 = si128;
2101 v437 = 4368;
2102 v438 = 18;
2103 v357 = 19;
2104 do
2105 {
2106 v353 ^= (v353 << 13) ^ ((v353 ^ (v353 << 13)) >> 7) ^ ((v353 ^ (v353 << 13) ^ ((v353 ^ (v353 << 13)) >> 7)) << 17);
2107 v358 = *((_BYTE *)&v435 + v357 + 7);
2108 *((_BYTE *)&v435 + v357 + 7) = v436.m128i_i8[v353 % v357];
2109 v436.m128i_i8[v353 % v357] = v358;
2110 v15 = v357-- == 1;
2111 }
2112 while ( !v15 && v357 != 1 );
2113 v359 = v355 ^ 0x53A9D1E794671459LL;
2114 v423 = si128;

```

여기 부분에서 permutation의 배열이 초기화가 되고 그 구조는 복원시

```
uint8_t opcodeTable1[19] = {
```

xmmword_140005200의 16바이트,

LOWBYTE(4368),

HIGHBYTE(4368),

18

```
};
```

위와 같고 4368 == 0x1110이므로 마지막 세 값은 리틀엔디안 기준으로 대략:

0x10, 0x11, 0x12이라는 걸 알 수 있고 따라서 xmmword_140005200의 앞 16바이트까지 IDA Hex View에서 추출하면 원본 19개 opcode 목록을 얻을 수 있다.

이를 위해 xmmword_140005200값을 추출하면

```

.rdata:00000001400051FC align 20h
.rdata:0000000140005200 xmmword_140005200 xmmword 0F0E0D0C0B0A09080706050403020100h

```

00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F이고 여기에 뒤의 3바이트를 붙히면 완전한 19바이트 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12를 구할 수 있다. 두 번째 배열 역시

```
v423 = si128;
```

```
v424 = 4368;
```

v425 = 18; 와 같이 실행되기 때문에 동일하게 시작한다.

그 다음 첫 번째 permutation을 분석을 하면 시작 위치는

```
2098  __mmword_7FF6A8055200 = 0;
2099  v353 = permutationNameState ^ 0x64C7920094671459LL;
```

다음과 같고 초기 배열의 설정은 아래와 같다.

```
v354 = v427.m128i_u32[0];
v355 = permutationNameState ^ ((unsigned __int64)v427.m128i_u32[0] << 32);
si128 = _mm_load_si128((const __m128i *)&__mmword_7FF6A8055200);
v436 = si128;
v437 = 4368;
v438 = 18;
v357 = 19;
```

그리고 실제 루프는

```
do
{
    v353 ^= (v353 << 13)
        ^ ((v353 ^ (v353 << 13)) >> 7)
        ^ ((v353 ^ (v353 << 13))
            ^ ((v353 ^ (v353 << 13)) >> 7)) << 17);

    v358 = *((_BYTE *)&v435 + v357 + 7);

    *((_BYTE *)&v435 + v357 + 7)
        = v436.m128i_i8[v353 % v357];

    v436.m128i_i8[v353 % v357] = v358;
```

```

v15 = v357-- == 1;
}

```

```
while ( !v15 && v357 != 1 );
```

이와 같은데(2114) 여기에서 ida윗부분의 변수 선언을 보면

v435 : rbp-200, v436 : rbp-1F8, v437 : rbp-1E8 , v438 : rbp-1E6 다음과 같고

```

2106 v357 = 19;
2107 do
2108 {
2109     v353 ^= (v353 << 13) ^ ((v353 ^ (v353 << 13)) >> 7) ^ ((v353 ^ (v353 << 13) ^ ((v353 ^ (v353 << 13)) >> 7)) << 17);
2110     v358 = *((_BYTE *)&v435 + v357 + 7);
2111     *((_BYTE *)&v435 + v357 + 7) = v436.m128i_i8[v353 % v357];
2112     v436.m128i_i8[v353 % v357] = v358;
2113     v15 = v357-- == 1;

```

첫 번째 루프에서는 v357 = 19이기 때문에 *((_BYTE *)&v435 + v357 + 7)이 주소 값은 계산시 &v435 + v357 + 7 = rbp-200 + 19 + 7 = rbp-200 + 26 = rbp-1E6이고 이는 v438의 주소다. 따라서 *((_BYTE *)&v435 + v357 + 7)는 v438을 가리키고 이 때 v436은

```

2102 si128 = _mm_load_si128((const __m128i *)&xmmword_7FF6A8055200);
2103 v436 = si128;

```

.rdata:00007FF6A8055200 xmmword_7FF6A8055200 xmmword 0F0E0D0C0B0A09080706050403020100h
.rdata:00007FF6A8055200 ; DATA XREF: start+263C↑

v436[0..15] = 배열 0~15이고 v437의 2바이트 = 배열 16~17이며 v438 = 배열 18 임으로 결국 v438은 table[18]과 같다.

그리고 v357이 감소하면 v357 = 18 → rbp-1E7 → table[17], v357 = 17 → rbp-1E8 → table[16]과 같이 내려가기 때문에 1차 permutation을 의사코드로 복원하면

```

for (i = 19; i > 1; i--)
{
    state = xorshift(state);

    temp = table[i - 1];
    table[i - 1] = table[state % i];
    table[state % i] = temp;
}

```

} 다음과 같은 의미를 지닌다.

이러한 정보를 이용해 첫 번째 permutation함수를 rvp4_permutation.py에 복원했다. 이는 source폴더에서 확인이 가능하다. 마찬가지로 2번째 permutation함수 역시 rvp4_permutation.py에 복원해 이제 그 파일에 이름이 JHON SMITH 이면 python rvp4_permutation.py "John Smith" --text-hash 19F8806B 와 같이 이름과 이전에 파악한 text-hash값 19F8806B을 입력하면

원본 이름 : 'John Smith'

정규화 이름 : 'john smith'

정규화 바이트 : 6A 6F 68 6E 20 73 6D 69 74 68

v298 : 0x13E1145C086C1907

v330 : 0x13E1145C9764BBE7

v334 : 0x1B803517D2705CD1

v419 : 0x1B803517C9F069C6

perm1 seed : 0x7F47A7175D977D9F

perm1 : 04 07 0D 05 0B 09 02 03 08 00 10 11 12 0C 0E 06 01 0F 0A

[논리] opcode -> perm1 opcode]

0x00 -> 0x04

0x01 -> 0x07

0x02 -> 0x0D

0x03 -> 0x05

0x04 -> 0x0B

0x05 -> 0x09

0x06 -> 0x02

0x07 -> 0x03

0x08 -> 0x08

0x09 -> 0x00

0x0A -> 0x10

0x0B -> 0x11

0x0C -> 0x12

0x0D -> 0x0C

0x0E -> 0x0E

0x0F -> 0x06

0x10 -> 0x01

0x11 -> 0x0F

0x12 -> 0x0A

v355 : 0x0278B57CC9F069C6

textSectionHash : 0x19F8806B

perm2 seed : 0x51D1649B5D977D9F

perm2 : 0A 0B 01 0E 00 07 0F 0D 02 12 10 03 09 08 04 06 05 11 0C

와 같이 opcode분석에 필요한 값들을 구할 수 있다.

그 다음엔 이 출력값들이 맞는지 확인하기 위해서 xdbg를 이용해 검증을 해 보도록 하겠다.

```

2144     break;
2145     v365 += byte_7FF68D6A51C0[v367];
2146     if ( v365 > 0xDA )
2147         break;
2148     byte_7FF68D6A6280[v366] = v436.m128i_u8[v367];
2149     if ( v365 == 218 )
2150         goto LABEL_448;

```

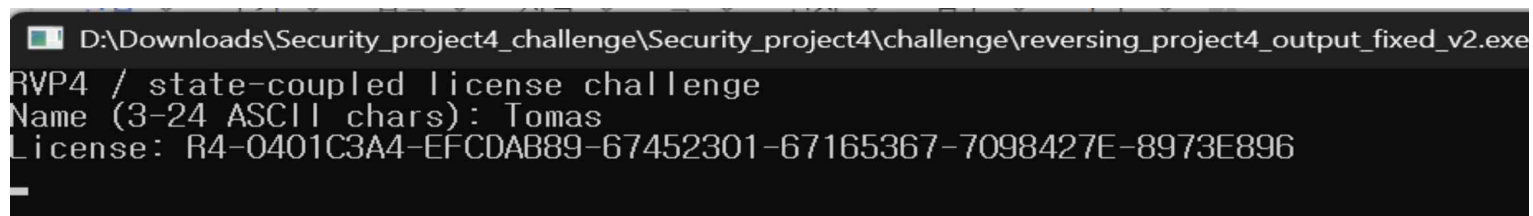
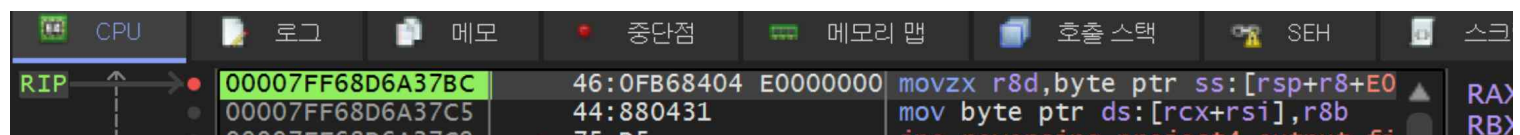


```

2162     v444 = v377 ^ 0xCEBEE4C49E1AF44LL;
2163     v368 = v423.m128i_i8[0];
2164     v369 = v423.m128i_i8[1];
2165     v370 = v423.m128i_i8[2];

```

위의 스크린샷에 나온 디컴파일된 코드 부분에 하드웨어 중단점을 걸고 살펴보면 이름 Tomas를 입력했을 때

이 지점에서 먼저 porem1값을 검증하기 위해서 덤프에서 rsp+e0의 위치로 이동해서 porem1값을 확인하면

주소	Hex	ASCII
0000005D942FFC	0C 08 11 07 06 0D 09 02 0B 04 03 00 01 10 0F 0E	
0000005D942FFC	05 12 0A 00 00 00 00 00 AB 00 00 00 CD 00 00 00	«...İ...
0000005D942FFC	80 96 98 00 00 00 00 00 00 00 00 00 00 00 00 00	

위에서 19비트인 0C 08 11 07 06 0D 09 02 0B 04 03 00 01 10 0F 0E 05 12 0A로 이는 visual studio에 rvp4_permutation.py의 입력으로 python rvp4_permutation.py Tomas --text-hash 19F8806B을 넣었을 때 나오는 값과 같다는 걸 아래의 스크린샷으로 알 수 있다.

```

PS C:\Users\정명훈> cd "D:\Downloads\Security_project4_challenge\Security_proje
ct4\challenge"
PS D:\Downloads\Security_project4_challenge\Security_project4\challenge> python
rvp4_permutation.py Tomas --text-hash 19F8806B
원본 이름      : 'Tomas'
정규화 이름    : 'tomas'
정규화 바이트  : 74 6F 6D 61 73
v298          : 0x59EF5B620806E136
v330          : 0x59EF5B60C77C3A26
v334          : 0x0B5860F65746A60A
v419          : 0x0B5860F65C1EC6FC
perm1 seed    : 0x6F9FF2F6C879D2A5
perm1         : 0C 08 11 07 06 0D 09 02 0B 04 03 00 01 10 0F 0E 05 12 0A

[논리 opcode -> perm1 opcode]
0x00 -> 0x0C

```

그 다음에 같은 방식으로 perm2를 검증하기 위해서

RIP →	0FB64424 50	movzx eax,byte ptr ss:[rsp+50]
	0FB65424 51	movzx edx,byte ptr ss:[rsp+51]
	44:0FB64424 52	movzx r8d,byte ptr ss:[rsp+52]

이 지점에 중단점을 설정하고 rsp + 50의 값을 덤프에서 확인해 보면 perm2값이

주소	Hex	ASCII
0000005D942FFC	08 07 0A 0F 12 0B 05 09 04 03 11 01 06 10 02 0E	
0000005D942FFC	0D 00 0C 00 00 00 00 00 00 00 00 00 00 00 00 00	
0000005D942FFC	16 00 00 00 FB 7F 00 00 6B 80 F8 19 FB 7F 00 00	û...k.ø.û...

위의 스크린샷에서처럼 08 07 0A 0F 12 0B 05 09 04 03 11 01 06 10 02 0E 0D 00 0C가 나오는데 이는 visual studio code에서의 perm2값과 같다는 걸 확인할 수 있다.

```

textSectionHash : 0x19F8806B
perm2 seed      : 0x4109317AC879D2A5
perm2           : 08 07 0A 0F 12 0B 05 09 04 03 11 01 06 10 02 0E 0D 00 0C
PS D:\Downloads\Security_project4_challenge\Security_project4\challenge>

```

이렇게 permutation에 대한 검증이 끝났으므로 그 다음에는 permutation 배열을 생성한 뒤 그 배열에 대한 참조를 따라가야 한다. 우선 perm1의 opcode가 사용되는 부분으로 가면 아래 스크린샷의 위치로 가게 되는데

```

2190     DWORD1(v448) = v377 + 1;
2191     v384 = byte_7FF68D6A6280[(unsigned int)v377];
2192     if ( v368 != v384 )
2193     {
2194         if ( v369 == v384 )
2195         {
2196             if ( (unsigned int)(v377 - 209) < 0xFFFFFFFF25 )
2197                 break;
2198             *(&v441 + (byte_7FF68D6A6280[v383] & 7)) = *(_QWORD *)&byte_7FF68D6A6280[(unsigned int)v377 + 2];
2199             LODWORD(v377) = v377 + 10;
2200             DWORD1(v448) = v377;
2201             goto LABEL_452;
2202         }
2203         if ( v370 == v384 )
2204         {
2205             v378 = byte_7FF68D6A6280[v383];
2206             LODWORD(v377) = v377 + 3;
2207             DWORD1(v448) = v377;
2208             v370 = byte_7FF68D6A6280[v383 + 21];

```

이 스크린샷의 코드를 분석해 보면 byte_7FF68D6A6280는 perm1과 관련된 내용을 가지고 있는데 v384에 opcode값을 perm1로 변환해 저장한 후에 이를 perm2의 각 원소값과 비교해서 일치한 perm2의 인덱스의 명령을 실행한다는 걸 알 수 있다. 이를 통해 최종적인 opcode mapping을 구할 수 있고 이것과 관련한 내용을 rvp4_permutation.py에 추가했으니 이를 확인하면 된다.

이를 통해 최종적인 opcode 복원하는 과정이 다 완료됐기 때문에 다음으로는 vm명령어 자체를 복원해야 한다. 이를 위해서 복잡한 if문을 하나씩 뜯어서 보면

먼저

```

2161 v443 = v355 ^ 0xCF881BF93044958CuLL;
2162 v444 = v377 ^ 0xCEBEE4C49E1AF44LL;
2163 v368 = v423.m128i_i8[0];
2164 v369 = v423.m128i_i8[1];
2165 v370 = v423.m128i_i8[2];
2166 v371 = v423.m128i_i8[3];
2167 v372 = v423.m128i_i8[4];
2168 v373 = v423.m128i_i8[5];
2169 v374 = v423.m128i_i8[6];
2170 v375 = v423.m128i_i8[7];
2171 v376 = v423.m128i_i8[8];
2172 v420 = v423.m128i_i8[10];
2173 LOBYTE(v431) = v423.m128i_i8[9];
2174 v432.m128i_i8[8] = v423.m128i_i8[12];
2175 v416 = v423.m128i_i8[11];
2176 v428.m128i_i8[8] = v423.m128i_i8[14];
2177 v432.m128i_i8[0] = v423.m128i_i8[13];
2178 v428.m128i_i8[0] = v423.m128i_i8[15];
2179 v421.m128i_i8[8] = v424;
2180 LODWORD(v377) = 0;
2181 v421.m128i_i8[0] = HIBYTE(v424);
2182 v427.m128i_i8[8] = v425;

v384 = byte_7FF68D6A6280[(unsigned int)v377];
if ( v368 != v384 )
{
    if ( v369 == v384 )
    {
        if ( (unsigned int)(v377 - 209) < 0xFFFFF25 )
            break;
        *(&v441 + (byte_7FF68D6A6280[v383] & 7)) = *(_QWORD *)&byte_7FF68D6A6280[(unsigned int)v377 + 2];
        LODWORD(v377) = v377 + 10;
        DWORD1(v448) = v377;
        goto LABEL_452;
    }
    if ( v370 == v384 )
    {
        v378 = byte_7FF68D6A6280[v383];

```

이 부분은 v369 = param2[1]와 opcode의 최종 변환이 같을 때 목적 레지스터이고 뒤의 8바이트는 상수이며 레지스터에 64비트 상수를 넣기 때문에 그러므로 param2[1]의 opcode기능은 MOV_IMM64라는 걸 알 수 있다.

그리고 v368 = param2[0]은 아무 것도 실행하지 않기 때문에 opcode의 기능은 nop이라는 걸 알 수 있다.

그 다음으로는 v370은 동적 분석을 통해서 어떤 기능을 수행하는지를 알아보려고 했으나,

여기에는 무서운 반전이 있으니 사실 이 변경된 코드는 license양식만 맞추면 늘 로그인이 되는 앞에서 분석한 내용이 전혀 쓸모가 없는 프로그램이었던 것이다. 하지만 다행이도 기존 원본 exe파일과 맨 마지막의

vm결과를 이용하는 부분만 다르기 때문에 최종 결과값만 원하는 값으로 변경시키는 걸 보여주면 곧 그것이 정답을 찾는 것이기 때문에 최종 vm결과를 원하는 대로 바꿔 보겠다.

```

2431 v414 = 0;
2432 for ( j = 0; j != 27; ++j )
2433 {
2434     v413 ^= (v413 << 13) ^ ((v413 ^ (v413 << 13)) >> 7) ^ ((v413 ^ (v413 << 13) ^ ((v413 ^ (v413 << 13)) >> 7)) << 17);
2435     *((_BYTE *)v459 + j) = byte_1400051E0[j] ^ (v413 >> (v414 & 0x38));
2436     v414 += 8;
2437 }
2438 v432 = 0;
2439 qword_1400060B0(qword_140006110, "Result: ", 8, &v432, 0);
2440 v432 = 0;
2441 qword_1400060B0(qword_140006110, v459, 27, &v432, 0);
2442 v432 = 0;
2443 qword_1400060B0(qword_140006110, "\r\n", 2, &v432, 0);
2444 return qword_1400060B8(0);
2445 }

```

0000337A start:2441 (140003F7A)

```
128 xmmword_140006240;
```

```
6A qword_140006278;
```

원본은 이렇게 v459값에 따라 성공 혹은 실패가 결정났지만

```

13 }
14 v422 = 0;
15 qword_7FF68D6A60B0(qword_7FF68D6A6110, "Result: ", 8, &v422, 0);
16 v422 = 0;
17 qword_7FF68D6A60B0(qword_7FF68D6A6110, "LICENSE ACCEPTED", 16, &v422, 0);
18 v422 = 0;
19 qword_7FF68D6A60B0(qword_7FF68D6A6110, "\r\n", 2, &v422, 0);
20 return qword_7FF68D6A60B8(0);
21 }

```

00000400 start:2411 (7FF68D6A1000)

스저된 버전은 license accepted가 나오는 문제가 있기 때문에 올바른 v459

```

0      break;
1      continue;
2  }
3  }
4  break;
5  }
6  v403 = v444 ^ v441 ^ __ROL8__(v442, 7) ^ __ROL8__(v443, 19);
7  v404 = 0;
8  for ( j = 0; j != 27; ++j )
9  {
10     v403 ^= (v403 << 13) ^ ((v403 ^ (v403 << 13)) >> 7) ^ ((v403 ^ (v403 << 13) ^ ((v403 ^ (v403 << 13)) >> 7)) << 17);
11     *((_BYTE *)v449 + j) = aLicenseAccepte[j] ^ (v403 >> (v404 & 0x38));
12     v404 += 8;
13 }

```

즉 수정된 버전 기준으로 올바른 v449를 만드는 걸 목표로 리버싱 과제의 목표를 변경 하겠다 이 때 v449는 4C 49 43 45 4E 53 45 20 41 43 43 45 50 54 45 44 20 20 20 20 20 20 20 20 20 즉 "LICENSE ACCEPTED" + 공백 11개가 되어야 한다. 그렇기 때문에 나의 목표도 v449가 그렇게 되게 만드는 것이다.

그렇게 V449를 만들기 위해서 V449가 만들어지는 과정을 봤더니

```

{
    v403 ^= (v403 << 13) ^ ((v403 ^ (v403 << 13)) >> 7) ^ ((v403 ^ (v403 << 13) ^ ((v403 ^ (v403 << 13)) >> 7)) << 17);
    *((_BYTE *)v449 + j) = aLicenseAccepte[j] ^ (v403 >> (v404 & 0x38));
    v404 += 8;
}

```

V403을 이용해 V449를 만든다는 걸 확인했고

$((_BYTE *)v449 + j) = aLicenseAccepte[j] \wedge (v403 \gg (v404 \& 0x38));$ 이 식에서 aLicenseAccepte가 뭔지를 조금 더 자세히 파악해 봤더니

.rdata:00007FF68D6A51E0 aLicenseAccepte db 'LICENSE ACCEPTED',0

즉 이게 내가 찾는 정답 "LICENSE ACCEPTED" + 공백 11개라는 걸 알 수 있었다.

그러면 위의 식에서 $(v403 \gg (v404 \& 0x38));$ 는 0이 되어야지 정답이 출력된다는 사실을 알 수 있다. 여기서 V404는 0에서 8씩 계속 증가하는 값이라는 걸 알 수 있고

v404 & 0x38를 for 루프를 돌린 값을 python코드로 확인하면

```
for i in range(0,27):
```

```
    print(hex((8*i) & 0x38))
```

이렇게 설정하고 돌리면

```
0x0. 0x8 ,0x10, 0x18, 0x20, 0x28, 0x30, 0x38
```

```
0x0. 0x8 ,0x10, 0x18, 0x20, 0x28, 0x30, 0x38
```

```
0x0. 0x8 ,0x10, 0x18, 0x20, 0x28, 0x30, 0x38
```

0x0, 0x8, 0x10 와 같이 나오고 내가 원하는 건 $(v403 \gg (v404 \& 0x38))$;는 0이 되어야 하기 때문에 결론은 v403이 0이 되어야 한다는 결론에 도달한다.

결국 이걸 영구패치해서 실행하면

```
D:\Downloads\Security_project4_challenge\Security_project4\challenge>reversing_project4_output_fixed_v3.exe
```

```
RVP4 / state-coupled license challenge
```

```
Name (3-24 ASCII chars): Tomas
```

```
License: R4-0401C3A4-EFCDAB89-67452301-67165367-7098427E-8973E896
```

```
Result: LICENSE ACCEPTED
```

나온다.

근데 이 프로젝트의 원본에도 사실 치명적인 문제가 있었는데 정상환경에서 실행해도 각종 변수에 따라서 정답이 안나오는 이상한 프로그램인 것이였던 것이다. 결국 더 이상 이 프로젝트를 진행하는 것이 의미가 많이 퇴색되어 정상적인 프로젝트였다면 나왔어야 되는 결과까진 얻었으니 여기에서 마무리를 하겠다.

배운점:

정석적인 리버싱 방법에 대해서 알아보고 각 변수의 이름을 맞게 설정하고 각 과정을 거치며 다양한 리버싱 기법에 대해서 알게 된 건 좋았다. 특히 하드웨어 중단점이 디버거 감지 우회에 쓰기 좋다는 걸 알게된 게 가장 좋았다.

아쉬운 점:

프로젝트를 ai한테 만들라고 시킬 때에는 늘 검증을 잘 하고 프로젝트의 완성도를 기반으로 평가를 해야 하는데 이 경우에는 내가 그냥 대충 믿고 넘어간 것이 화근이 되어 원본 코드는 정상적인 실행에 대해서도 너무 복잡한 절차를 역어 정상적인 결과가 나오지 못했고 수정된 코드로 실행된 exe파일은 입력의 형식만 맞추면 정답이 나오게 되는 등 프로젝트 시작시 프로젝트의 완성도 평가를 ai를 너무 믿고 적당히 넘어갔던게 문제가 됐다 다음에는 철저히 검증을 통해 프로젝트 자체의 모순과 오류를 검증하고 프로젝트를 진행하겠다.